Estándares de API — Backend Rails

Fecha: 2026-04-11
Aplica a: Todos los controllers en `app/controllers/api/v1/`

---

1. Estructura de un Controller

Todo controller sigue esta estructura exacta. No agregar métodos públicos fuera de las acciones REST.

```ruby
# frozen_string_literal: true

class Api::V1::RecursosController < Api::V1::BaseController
  before_action :set_recurso, only: [:show, :update, :destroy]

  def index   ; end
  def show    ; end
  def create  ; end
  def update  ; end
  def destroy ; end

  private

  def set_recurso
    @recurso = Recurso.find(params[:id])
  rescue ActiveRecord::RecordNotFound
    render json: { error: ERR_NOT_FOUND }, status: :not_found
  end

  def recurso_params
    params.require(:recurso).permit(:campo1, :campo2)
  end
end
```

Reglas:

- Siempre `# frozen_string_literal: true`
- `set_recurso` **siempre** usa `rescue ActiveRecord::RecordNotFound` con `ERR_NOT_FOUND`
- `before_action :set_recurso` solo en acciones que necesitan el registro: `[:show, :update, :destroy]`
- Acciones extra (no-CRUD) van en la sección `collection` o `member` de las rutas, nunca inventadas

---

2. Formato de Respuestas

Recurso único — éxito

```
# show / create / update
render json: @recurso                                  # simple
render json: serialize_recurso(@recurso)               # con helper privado
render json: RecursoSerializer.new(@recurso).as_json # con serializer
```

Colección con paginación

El formato estándar para índices paginados. **Siempre incluir `meta`.**

```
render json: {
  data: @recursos.map { |r| serialize_recurso(r) },
  meta: {
    current_page: page,
    per_page:     per_page,
    total_count:  total_count,
    total_pages:  (total_count.to_f / per_page).ceil
  }
}
```

> **Inconsistencia actual conocida:** algunos controllers usan `data/meta`, otros usan `pagination`. Nuevo código usa `data/meta`. Al tocar un controller viejo, migrar al mismo tiempo.

Error de validación

```
render json: { errors: @recurso.errors.full_messages }, status:
:unprocessable_content
```

Siempre array, nunca string. El FE hace `error.response.data.errors.join(', ')`.

Error de negocio (no de validación)

```
render json: { error: 'Mensaje explicativo' }, status:
:unprocessable_content
# o
render json: { error: ERR_NOT_FOUND }, status: :not_found
```

Constantes definidas en `BaseController`:

Constante	Mensaje
`ERR_NOT_FOUND`	`'Recurso no encontrado'`
`ERR_UNAUTHORIZED`	`'Sesión expirada o inválida (No JWT found)'`
`ERR_INVALID_TOKEN`	`'Token inválido'`

Constante	Mensaje
`ERR_EXPIRED_TOKEN`	`'Token expirado'`
`ERR_REVOKED_TOKEN`	`'Token revocado'`
`ERR_CREDENTIALS`	`'Credenciales inválidas'`

Delete exitoso

```
head :no_content  # 204, sin body
```

---

3. Paginación

Parámetros estándar que el FE envía:

Parámetro	Default	Descripción
`page`	`1`	Página solicitada
`per_page`	`20` ó `50`	Registros por página

```
page     = params[:page]&.to_i     || 1
per_page = params[:per_page]&.to_i || 20

# Con kaminari (preferido para queries complejas):
@records = scope.page(page).per(per_page)
total    = scope.count

# Con limit/offset (para queries simples):
@records = scope.limit(per_page).offset((page - 1) * per_page)
total    = scope.count
```

Sin paginación: cuando se usan para dropdowns, limitar siempre a 500 con `.limit(500)`. Nunca retornar todos los registros sin límite.

---

4. Filtros

Los filtros se aplican progresivamente sobre el scope. Cada filtro es opcional e independiente.

```
def index
  scope = Recurso.includes(:asociacion)

  # Filtro por ID foráneo
  scope = scope.where(client_id: params[:client_id]) if
params[:client_id].present?
```

```
  # Filtro boolean
  scope = scope.where(status: params[:status] == 'true') if
params[:status].present?

  # Filtro por rango de fechas (patrón estándar)
  if params[:fecha_inicio].present? && params[:fecha_fin].present?
    scope = scope.where(fecha: params[:fecha_inicio]..params[:fecha_fin])
  elsif params[:fecha_inicio].present?
    scope = scope.where('fecha >= ?', params[:fecha_inicio])
  elsif params[:fecha_fin].present?
    scope = scope.where('fecha <= ?', params[:fecha_fin])
  end

  # Búsqueda de texto — siempre ILIKE (case-insensitive), nunca LIKE
  if params[:search].present?
    term = "%#{params[:search]}%"
    scope = scope.where('campo ILIKE ? OR otro_campo ILIKE ?', term, term)
  end

  scope = scope.order(created_at: :desc)
  # ... paginación y render
end
```

Reglas de filtros:

- Siempre `params[:x].present?` antes de aplicar — nunca `if params[:x]`
- Búsquedas de texto: `ILIKE` para case-insensitive (PostgreSQL)
- Búsqueda por nombre completo: `CONCAT(nombre, ' ', apaterno, ' ', amaterno) ILIKE ?`
- Joins para filtrar por asociaciones: preferir `.joins` sobre `.includes` cuando solo se filtra (no se serializa)

---

5. Eager Loading (N+1)

Regla: **nunca** hacer una query dentro de un `.map` o bucle.

```
# ❌ MAL — N+1 queries
@bookings.map { |b| { cliente: b.client.razon_social } }

# ✅ BIEN — 1 query con includes
@bookings = TtpnBooking.includes(:client, :vehicle, :employee)
@bookings.map { |b| { cliente: b.client&.razon_social } }
```

Patrón de serialización en el controller:

Para recursos sin serializer dedicado, usar un método `serialize_x` privado en el controller:

```
private

def serialize_gas_charge(charge, detailed: false)
  base = {
    id:          charge.id,
    vehicle_clv: charge.vehicle&.clv,  # safe navigation siempre
    monto:       charge.monto,
    fecha:       charge.fecha,
    hora:        charge.hora&.strftime('%H:%M')
  }
  base.merge!(lat: charge.lat, lng: charge.lng) if detailed
  base
end
```

Para modelos complejos con muchos campos o vistas diferentes (minimal vs full), usar un Serializer:

```
# app/serializers/employee_serializer.rb
class EmployeeSerializer
  def initialize(employee, minimal: false)
    @e = employee
    @minimal = minimal
  end

  def as_json
    @minimal ? minimal_json : full_json
  end
  # ...
end
```

---

6. Filtrado por Business Unit

Todos los controllers que devuelven datos de empleados, vehículos, clientes, etc. deben respetar la Business Unit del usuario.

```
# Opción A — scope en el modelo (preferida para modelos con scope
definido)
@employees = Employee.business_unit_filter.where(...)

# Opción B — filtro explícito (para modelos sin scope)
if @business_unit_id
  scope = scope.where(business_unit_id: @business_unit_id)
end
# @business_unit_id viene de BaseController#set_business_unit_id
# Es nil si el usuario es sadmin (puede ver todo)
```

---

7. Acciones Extra (no-CRUD)

Las acciones que no son index/show/create/update/destroy se declaran en las rutas así:

```
# Para acciones sobre un registro específico (member):
resources :employees do
  member do
    patch :activate    # PATCH /api/v1/employees/:id/activate
    patch :deactivate  # PATCH /api/v1/employees/:id/deactivate
  end
end

# Para acciones sobre la colección (collection):
resources :ttpn_bookings do
  collection do
    get  :stats   # GET /api/v1/ttpn_bookings/stats
    post :import  # POST /api/v1/ttpn_bookings/import
  end
end
```

En el controller, estas acciones son métodos públicos normales:

```
def activate
  @employee.update(status: true)
  render json: EmployeeSerializer.new(@employee, minimal: true).as_json
end
```

---

8. Jobs Asíncronos (Sidekiq)

Cuando una acción tarda más de ~2 segundos, usar un job. El controller retorna el `job_id` inmediatamente.

```
def import
  job_id = TtpnBookingImportJob.perform_async(params[:file],
current_user.id)
  render json: { job_id: job_id, message: 'Procesando...' }, status:
:accepted
end

# Endpoint de status que el FE consulta periódicamente:
def import_status
  status = Sidekiq::Status::status(params[:job_id])
  pct    = Sidekiq::Status::pct_complete(params[:job_id])
  render json: { status: status, progress: pct }
end
```

Colas disponibles:

Cola	Uso
`default`	Jobs generales
`payrolls`	Cálculo de nómina (jobs pesados)
`alerts`	Dispatch de alertas

---

9. Rutas — Convenciones de Nombres

- Recursos: **snake_case plural** → `travel_counts`, `ttpn_bookings`, `gas_charges`
- Namespace API: siempre bajo `api/v1`
- Las rutas están divididas por dominio en `config/routes/`:
  - `auth.rb`, `vehicles.rb`, `employees.rb`, `clients.rb`, `bookings.rb`, `payroll.rb`, `fuel.rb`, `administration.rb`, `alerts.rb`, `dashboard.rb`
- Agregar recursos nuevos al archivo de dominio correspondiente, no a `routes.rb` directamente

---

10. Params — Whitelist

Nunca `params.permit!`. Siempre whitelist explícita:

```ruby
def recurso_params
  params.require(:recurso).permit(
    :campo1, :campo2,
    nested_attributes: [:id, :campo, :_destroy]
  )
end
```

El `require(:recurso)` corresponde al wrapper que envía el FE: `{ recurso: { campo1: ... } }`.

---

11. Auditoría (created_by / updated_by)

Los modelos que tienen `created_by_id` / `updated_by_id` se llenan en el controller, no en el modelo:

```ruby
def create
  @recurso = Recurso.new(recurso_params)
  @recurso.created_by_id = current_user.id
  # ...
end

def update
  @recurso.updated_by_id = current_user.id
  @recurso.update(recurso_params)
  # ...
end
```

Para campos legacy con nombre de usuario en texto (`created_by` string):

```
@gas_charge.created_by = current_user&.nombre || 'API'
```